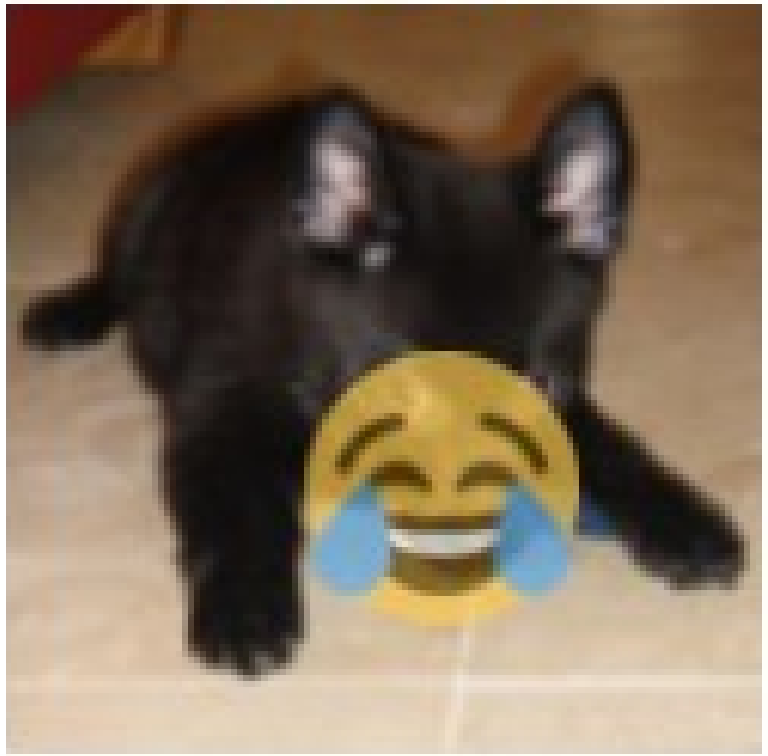
CS 229: HW 2 Confident Emoji Hunters



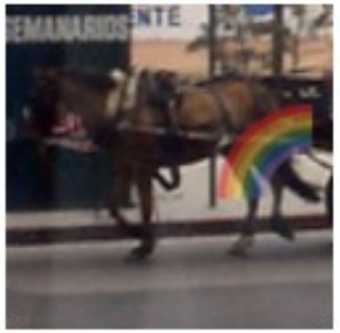
Spot the emoji – how confident is your neural net?

emoji

emoji

emoji

emoji



emoji

emoji

emoji

clean

clean

clean

clean

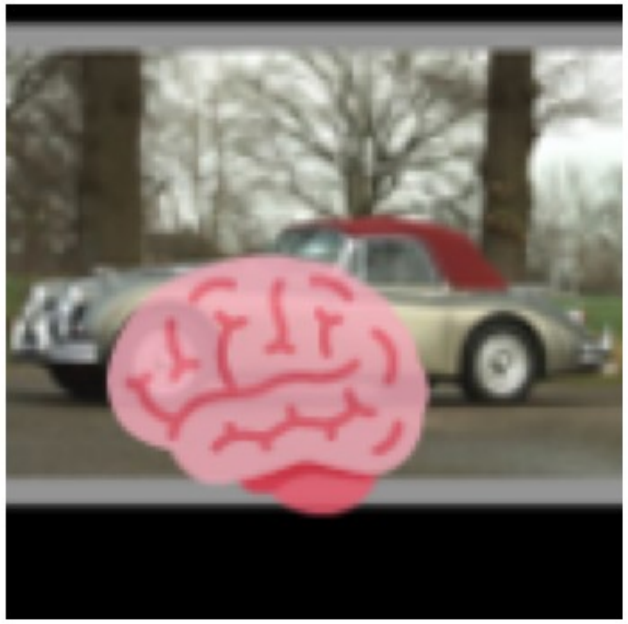
clean

clean

clean

E

conf=1.000 (correct)

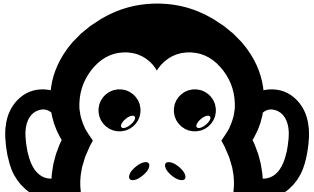


Neural net
is confident
it’s an emoji

conf=0.517 (WRONG)

Not
confident
it’s an emoji

(He’s so cute he could be an emoji!)



Google AI
Brain Team

How do we measure uncertainty?

$$\text{Calibration Error} = |\text{Confidence} - \text{Accuracy}|$$

predicted probability of correctness

observed frequency of correctness

How exactly are these defined? We have different answers for binary and multi-class.

Confidence in a particular outcome (binary – outcome happens or not)

Will it rain (Y=1), or not (Y=0)
Is there an emoji (Y=1), or not (Y=0)

Predicted probability the outcome happens

"Confidence" $= P_{model}\ (Y = 1)$

Observed frequency the outcome happens

"Accuracy" $= P_{data}(Y = 1)$

Always group in bins with same prediction probability / confidence

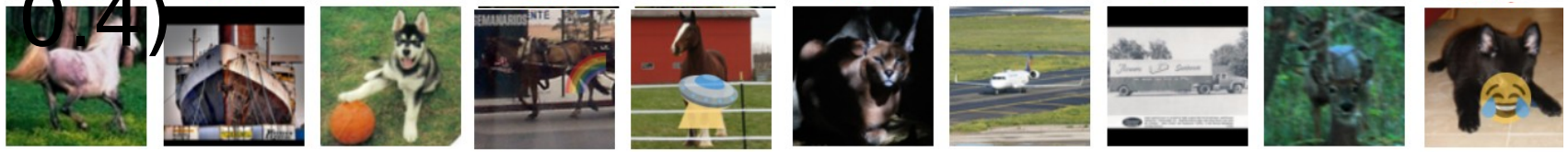
True label	0	0	0	0	0	0	0	1	1	1	Calibrated?
Model prediction	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	0.3	

True label	1	0	1	1	0	1	1	0	1	1	Calibrated?
Model prediction	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	

True label	1	0	1	1	0	1	1	0	1	1	Calibrated?
Model prediction	0.7	0.7	0.7	0.7	0.7	0.7	0.7	0.7	0.7	0.7	

ECE?

Slightly more realistic – a bin has some range, e.g. [0.3, 0.4)



True label	0	0	0	1	1	0	0	0	0	1	Calibration error?
Model prediction	0.31	0.39	0.33	0.35	0.37	0.35	0.32	0.30	0.39	0.32	?

We use the average confidence: 0.343
And compare to average frequency: 0.3

Calibration error (in the [0.3, 0.4) bin) is 0.043

Multi-class confidence

Does the predicted probability truly reflect the probability of making a correct prediction?

- **Confidence** = maximum class probability predicted by classifier

$$\begin{pmatrix} p(cat) = 0.7 \\ p(dog) = 0.2 \\ p(bird) = 0.1 \end{pmatrix}$$

- **Confidence calibration:**

$$P(\text{correct}|\text{confidence} = p) = p \qquad \forall p \in [0, 1]$$

e.g., predictions with confidence of 90% should be correct 90% of the time.

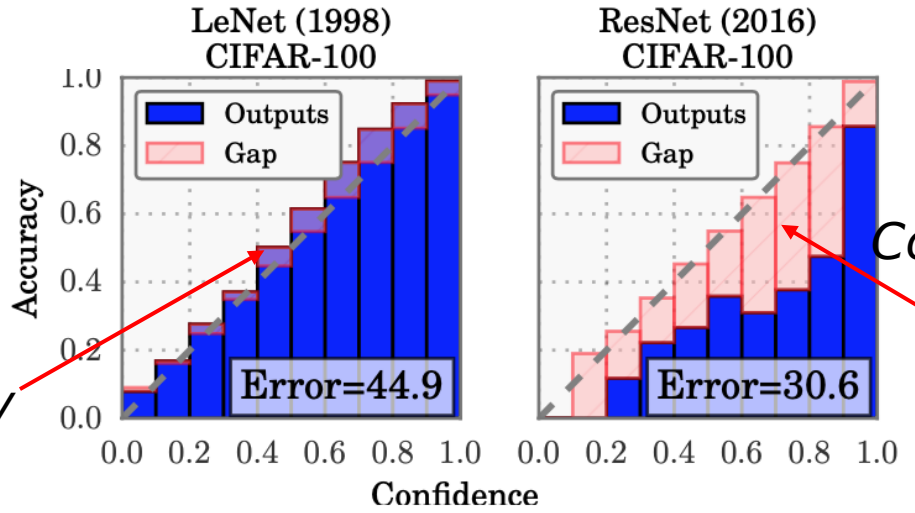
This paper argues that binary definition may be better to use even in multi-class cases:

Canonical “Guo et al 2017” paper also has both definitions

Expected Calibration Error [Naeini+ 2015]:

Image source: Guo+ 2017 “On calibration of modern neural networks”

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left|\mathrm{acc}(b) - \mathrm{conf}(b)\right|$$



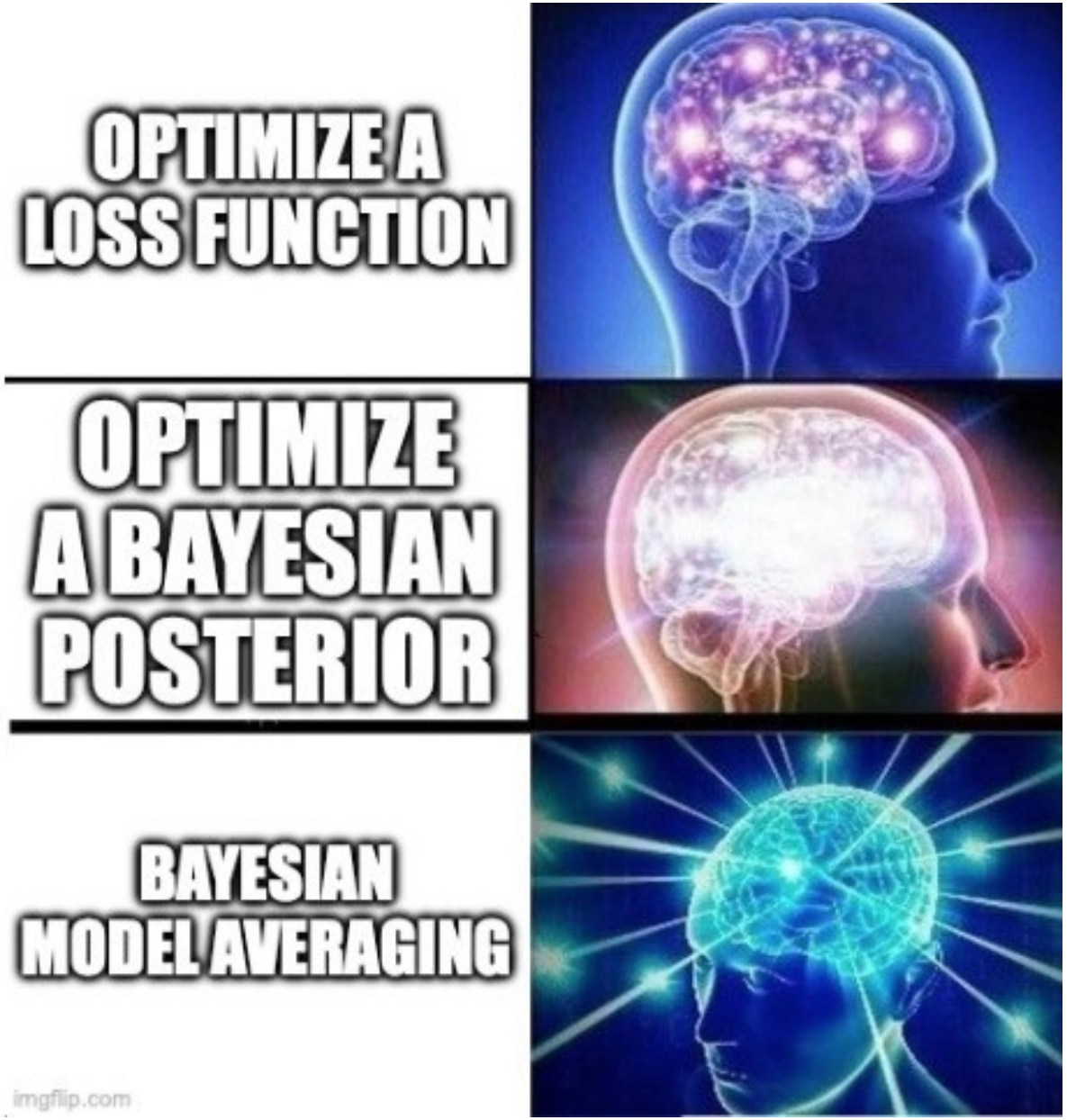
Confidence < Accuracy

=> Underconfident

Confidence > Accuracy

=> Overconfident

Deep Bayes and Uncertainty



Bayesian DL Paper links

For longer background on Bayesianism – Go to: Canvas / Yuja / CS_224 lectures / March 6 and March 8

- Deep Ensembles,
- Bayesian deep learning (Multi-SWAG)
- Bayesian Skeptics
- and Response,
- Bayesian model selection

I was very influenced by the work of Pavel Izmailov and Andrew Gordon Wilson
https://www.youtube.com/watch?v=QqmYD6TzcrA

https://www.youtube.com/watch?v=lhwk4ESlyMA

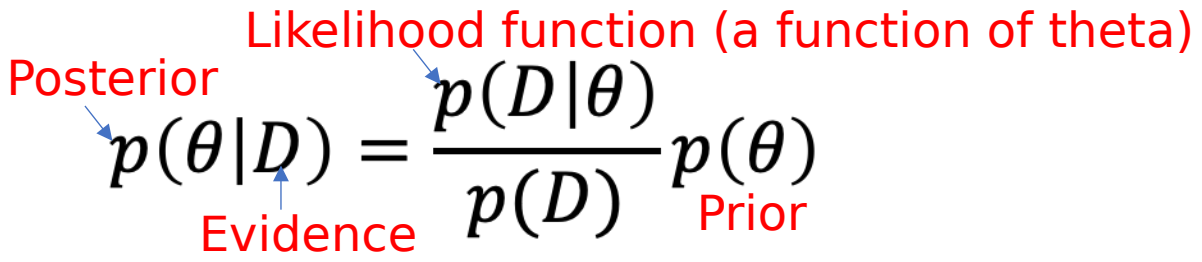
Bayesianism

- Probability is all in your head
- Predictions use the full probability distribution of your beliefs (as opposed to optimization – a single best guess)

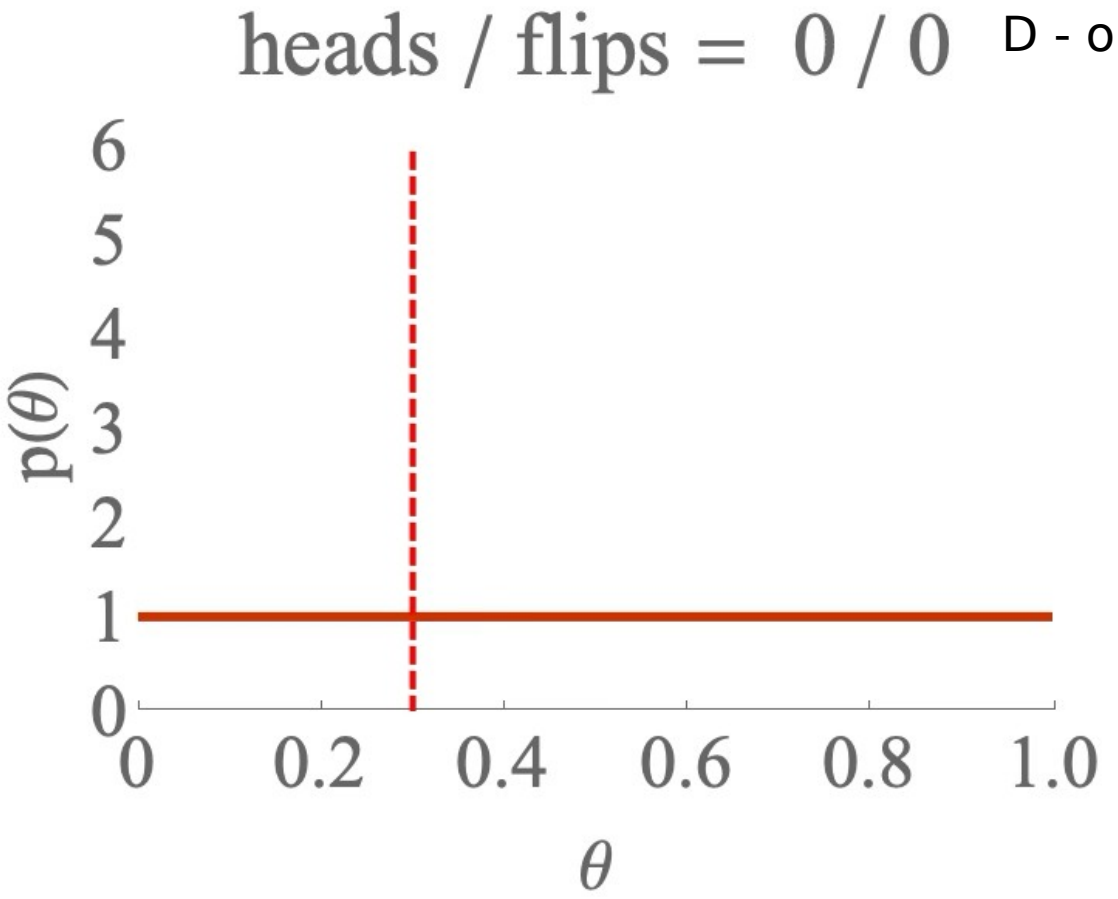
Interpret Bayes rule as a Bayesian

ϑ – Hypotheses (e.g. the weight of the coin or neural net weights)

D – data/evidence (y1=Tails, y2=Heads,….) (or labeled images, etc.)



Theta is the probability of heads
The ground truth for this example is:
θ* = 0.3



D - our data

D =
{0,0,0,0,0,0,0,1,0,1,1,0,0
,1,1,0,0,0,0,0,0,0,1,0,0,
0,0,0,0,0,0,0,0,0,0,0,1,
0,1,1,0,1,0,0,0,0,0,1,0,0,
0,1,0,1,0,1,1,0,0,1,1,0,1,
1,0,1,1,1,1,0,0,0,0,0,0,1,
0,1,1,0,0,0,0,0,0,0,1,0,1,
0,1,0,0,1,1,1,0,0}

Our prior… picked out of thin air!

$$p(D|\theta) = \prod_i p(y_i = heads|\theta) = \prod_i \theta^{y_i}(1-\theta)^{1-y_i}$$

Likelihood to observe data

What's our subjective probability of a heads on the next flip, given the data we have seen?

$$p(D|\theta) = \prod_i p(y_i = heads|\theta) = \prod_i \theta^{y_i}(1-\theta)^{1-y_i}$$

Posterior on parameters given data

$$p(\theta|D) = \frac{\prod_i \theta^{y_i}(1-\theta)^{1-y_i}}{Z(D)} p(\theta)$$

- Ok, but what's our best *prediction for the next y?*
- Maximum a Posteriori (MAP)

$$\theta_{MAP} = \arg\max_\theta p(\theta|D)$$

$$p(y = heads|\theta_{MAP}) = \theta_{MAP}$$

Bayesian model averaging

- MAP is *a little* Bayesian, because we use the prior – often we speak of regularizers as imposing Bayesian priors, but we're not using the whole posterior

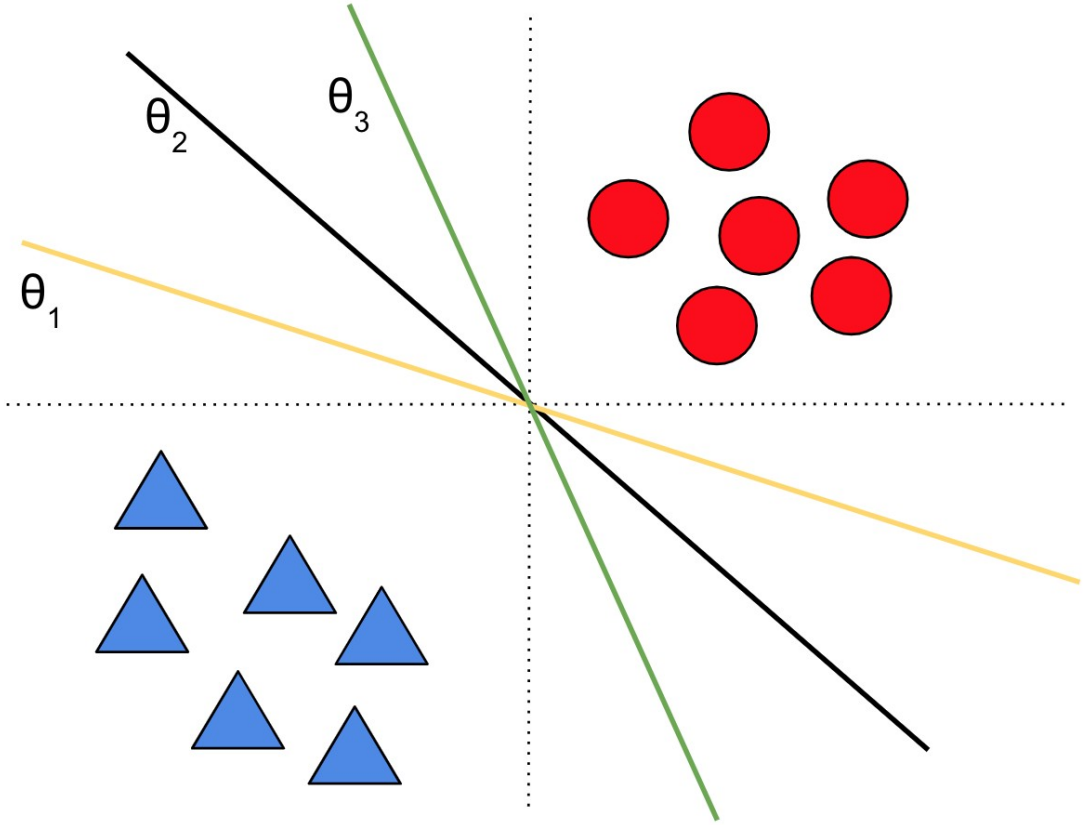
- The fully Bayesian way

$$p(y = heads|D) = \int p(y = heads|\theta)p(\theta|D)d\theta$$

Integrate over *all possible models weighted according to our belief*

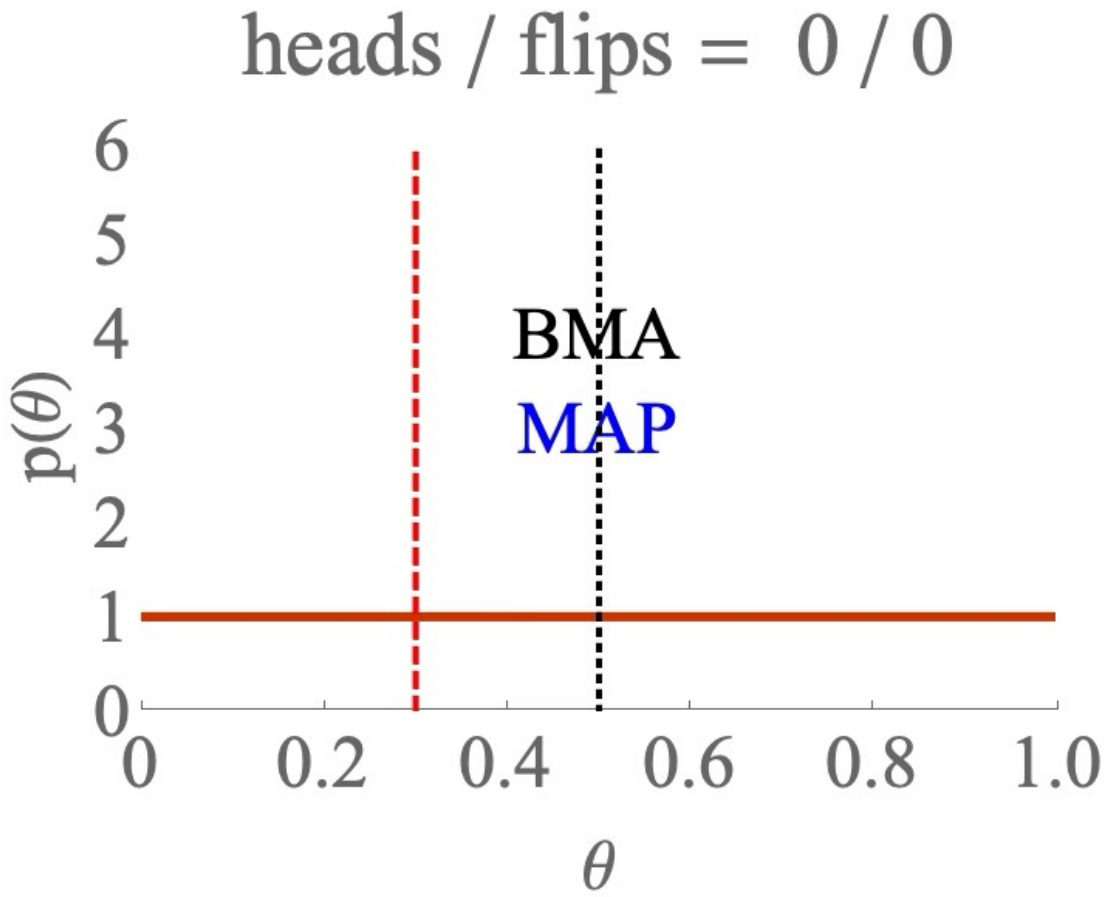
Obviously, not so easy in general. (Super easy in this case – Dirichlet distribution and conjugate priors)

Connection with uncertainty…



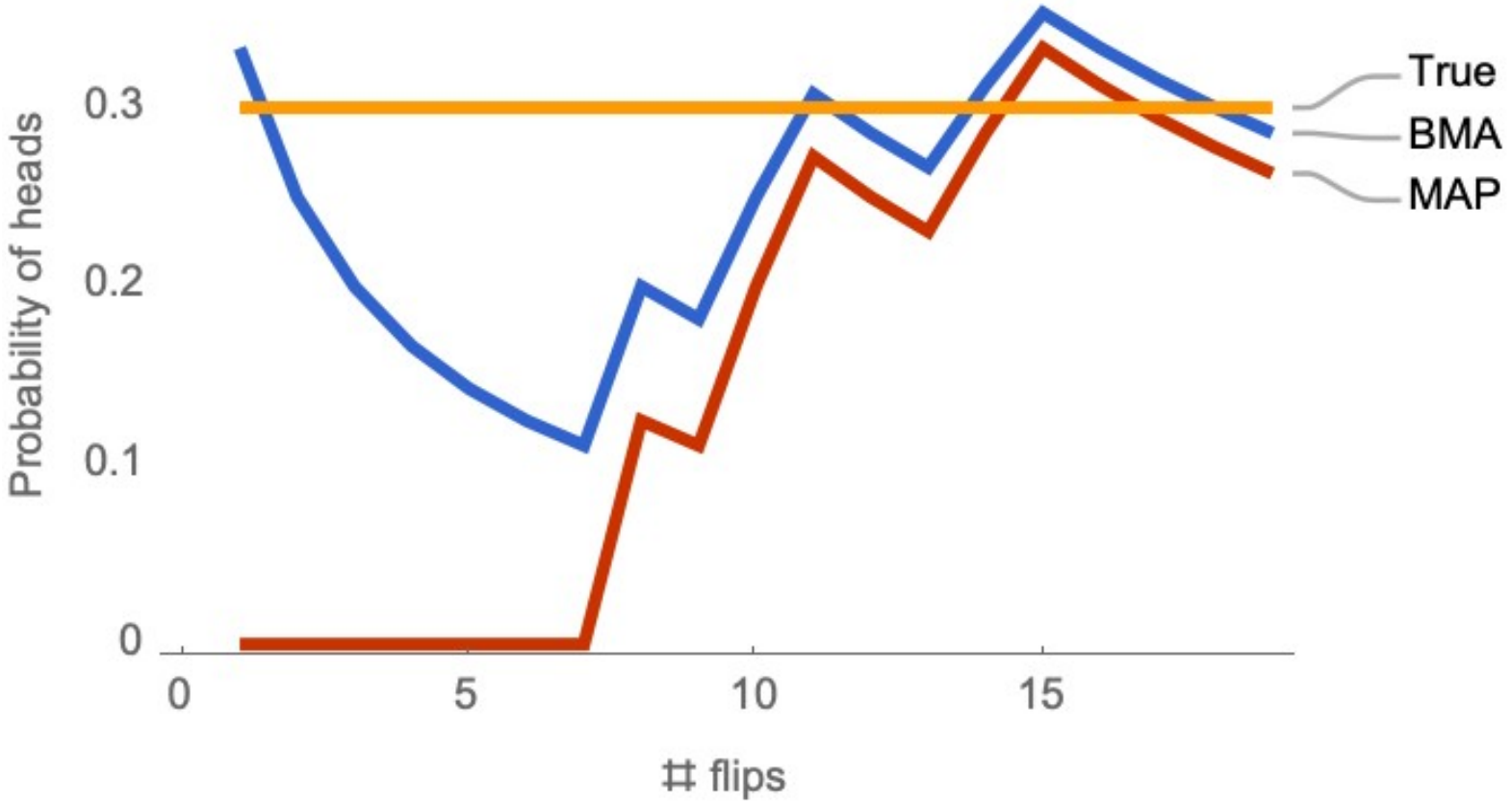
$$p(y = heads|D) = \int p(y = heads|\theta)p(\theta|D)d\theta$$

$$p(y = blue|x, D) = \int p(y = blue|x, \theta)p(\theta|D)d\theta$$



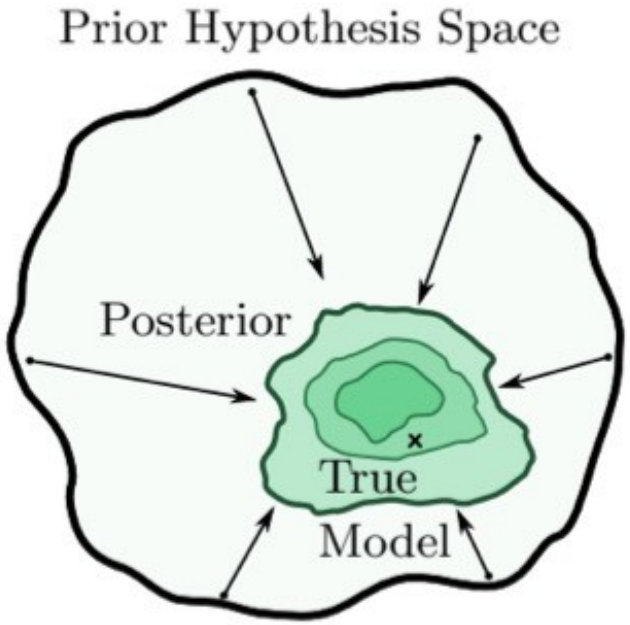
D =

{0,0,0,0,0,0,0,1,0,1,1,0,0
,1,1,0,0,0,0,0,0,0,1,0,0,
0,0,0,0,0,0,0,0,0,0,0,1,
0,1,1,0,1,0,0,0,0,0,1,0,0,
0,1,0,1,0,1,1,0,0,1,1,0,1,
1,0,1,1,1,1,0,0,0,0,0,0,1,
0,1,1,0,0,0,0,0,0,0,1,0,1,
0,1,0,0,1,1,1,0,0}

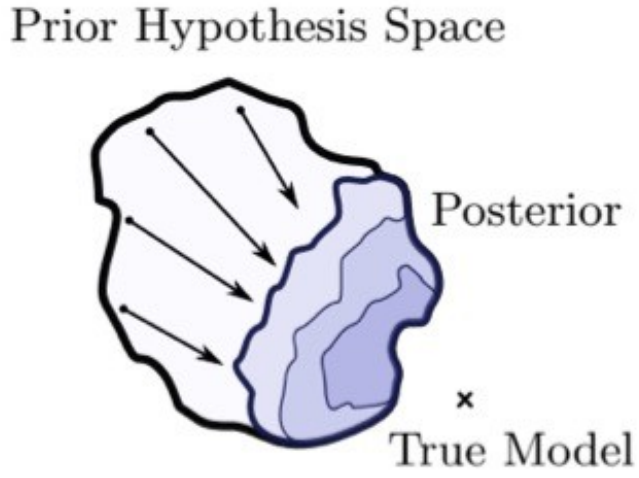


D =T, T, T, T, T, T, T, H, T,

Extending our Bayesian intuition to higher dimensions / neural nets

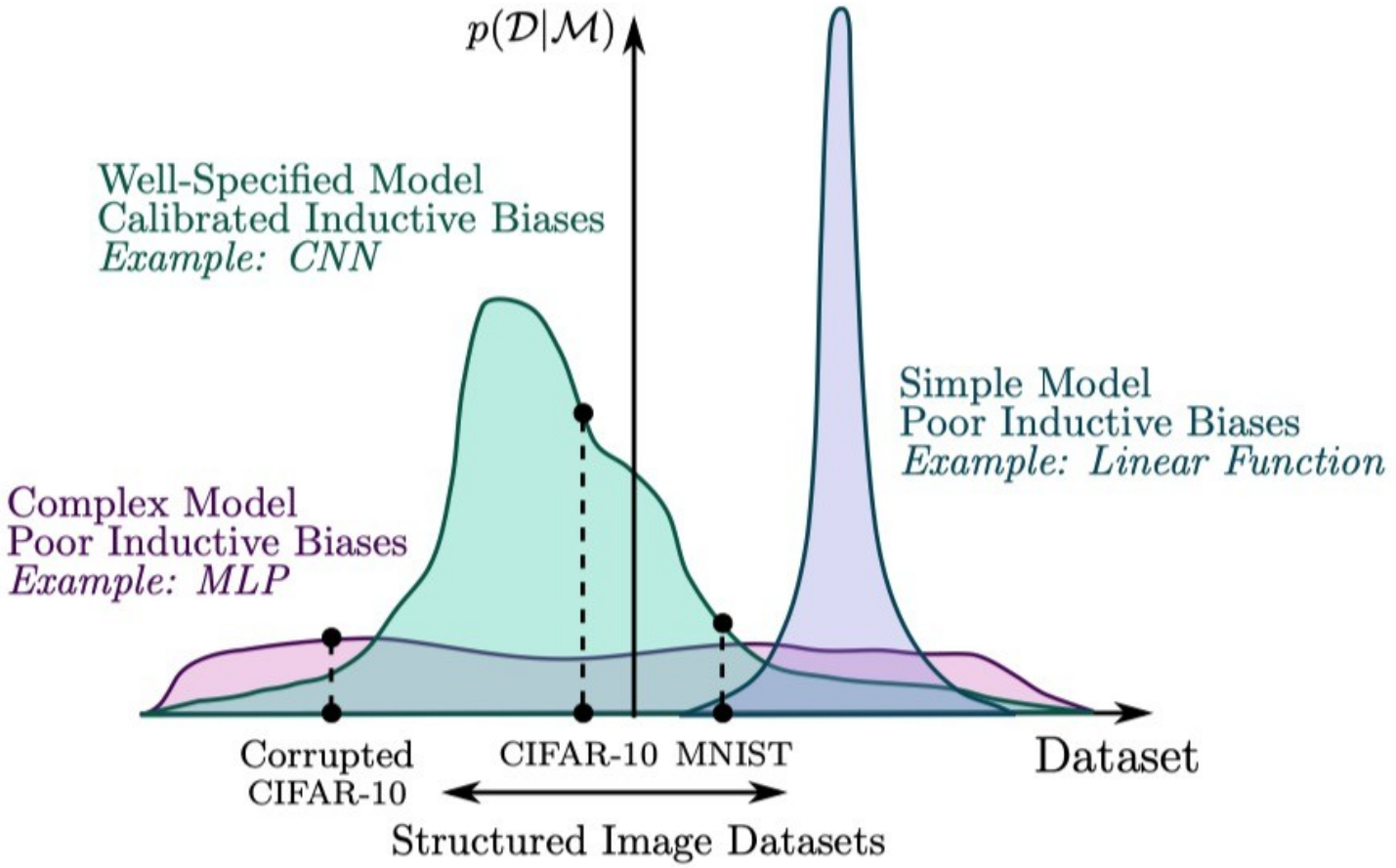


Large hypothesis space includes true model – contracts to the correct distribution with enough data



Small hypothesis space – mis-specified!

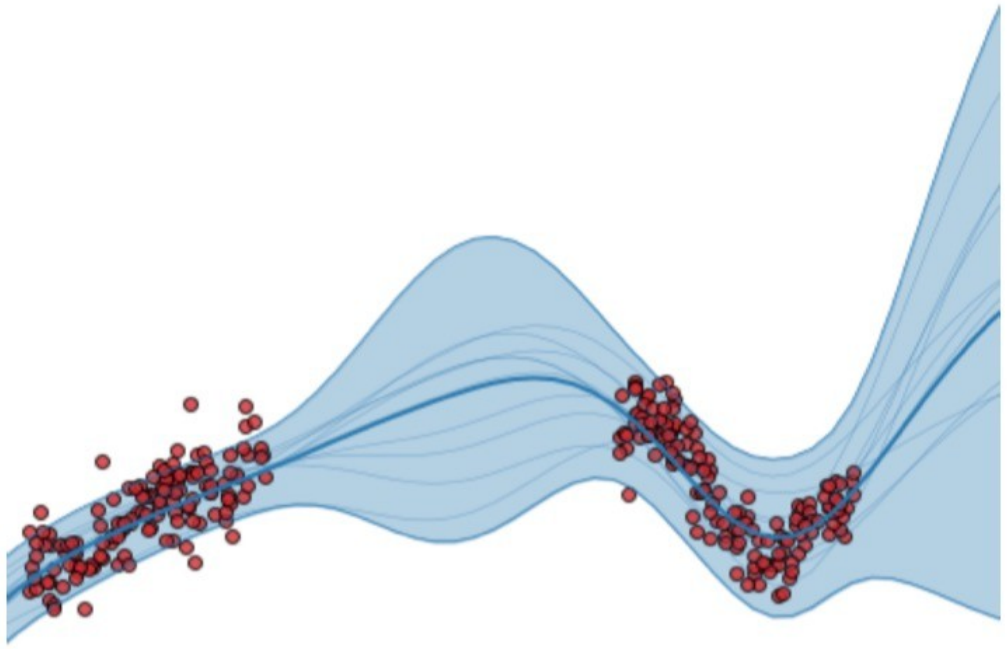
How likely are image datasets under different “priors” or model classes?



Bayesian regression

The key property of a Bayesian approach:

- We use the entire *posterior distribution* of parameters, $\boldsymbol{\theta}$, given data
- We use **marginalization** (over $\boldsymbol{\theta}$) instead of optimization



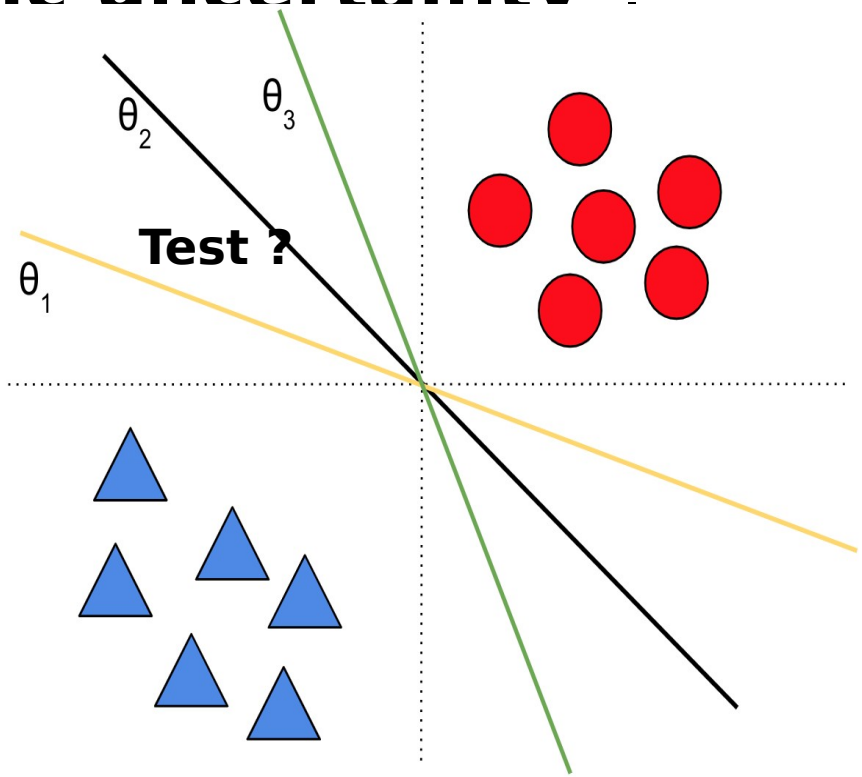
Bayesian model averaging
(for classification or regression)

$$p(y|x, D) = \int p(y|x, \theta)p(\theta|D)d\theta$$

Especially important in deep learning!

In practical DL, there are many equally good, but different, solutions.

Hence BMA deals with our "**epistemic uncertainty**".



Deep Ensembles: A Loss Landscape Perspective

Stanislav Fort*
Google Research
sfort1@stanford.edu

Huiyi Hu*
DeepMind
clarahu@google.com

Balaji Lakshminarayanan†
DeepMind
balajiln@google.com

Especially important in deep learning!

In practical DL, there are many equally good, but different, solutions.

Hence BMA deals with our "**epistemic uncertainty**".

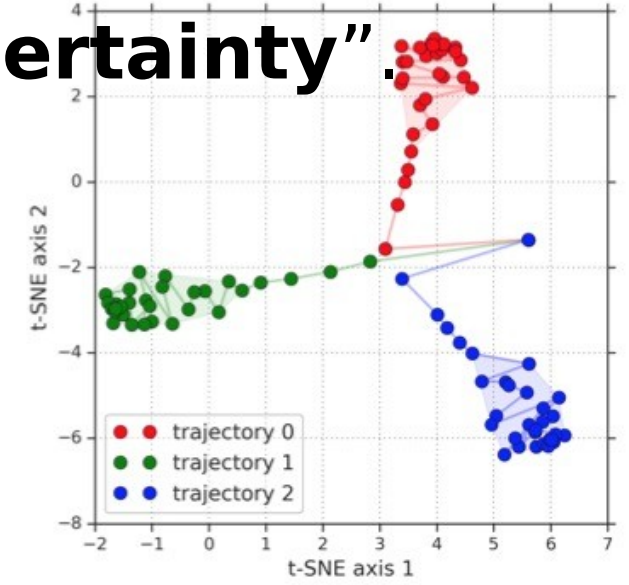
Deep Ensembles: A Loss Landscape Perspective

Stanislav Fort* Google Research sfort1@stanford.edu

Huiyi Hu* DeepMind clarahu@google.com

Balaji Lakshminarayanan† DeepMind balajiln@google.com

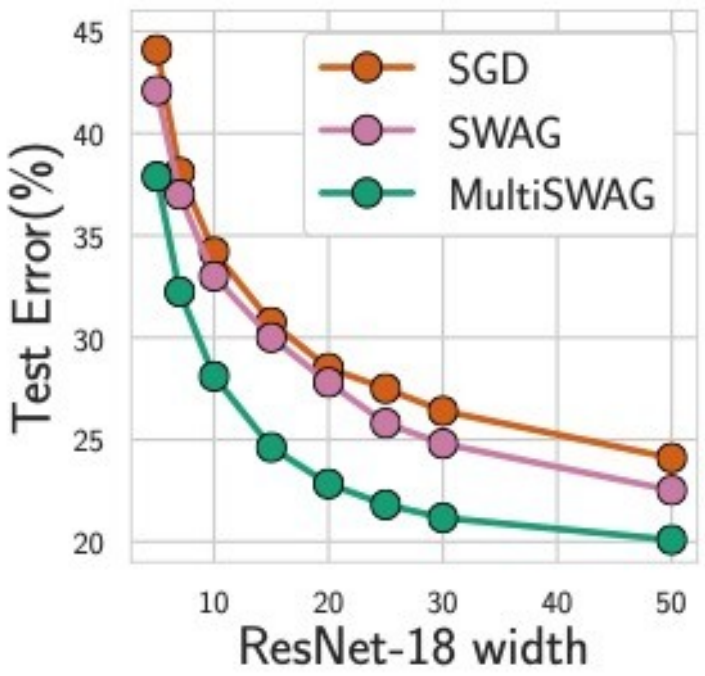
When we retrain Neural Nets: Predictions on test are equally good *on average*, but different samples are correct/incorrect in each model!



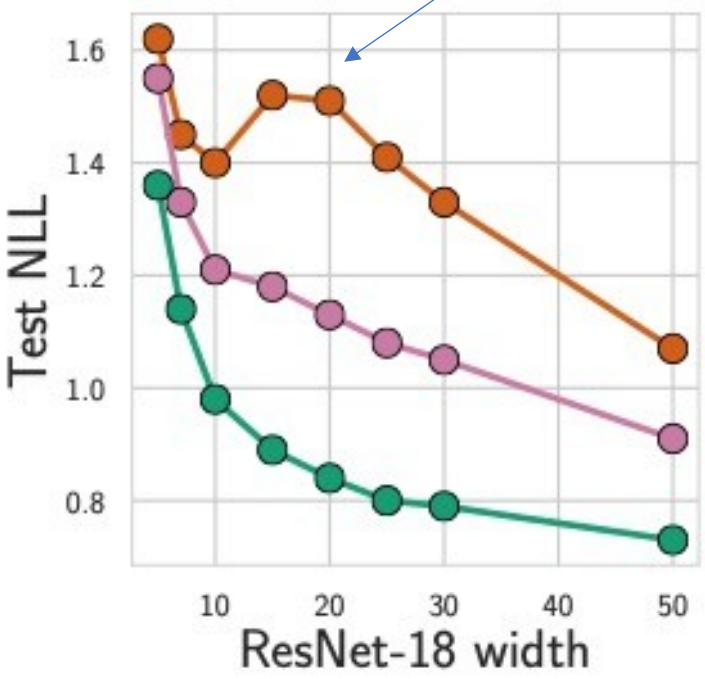
(c) t-SNE of predictions

Example benefits of (practical) Bayesian approaches

Double descent for SGD eliminated!

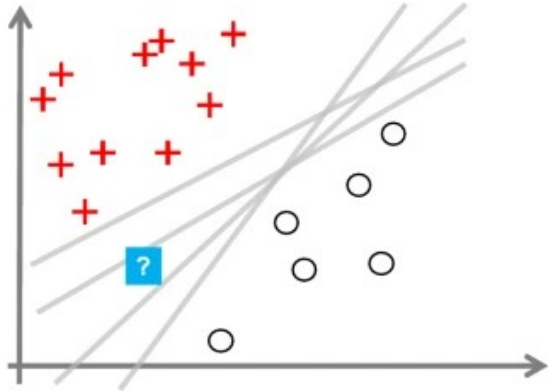


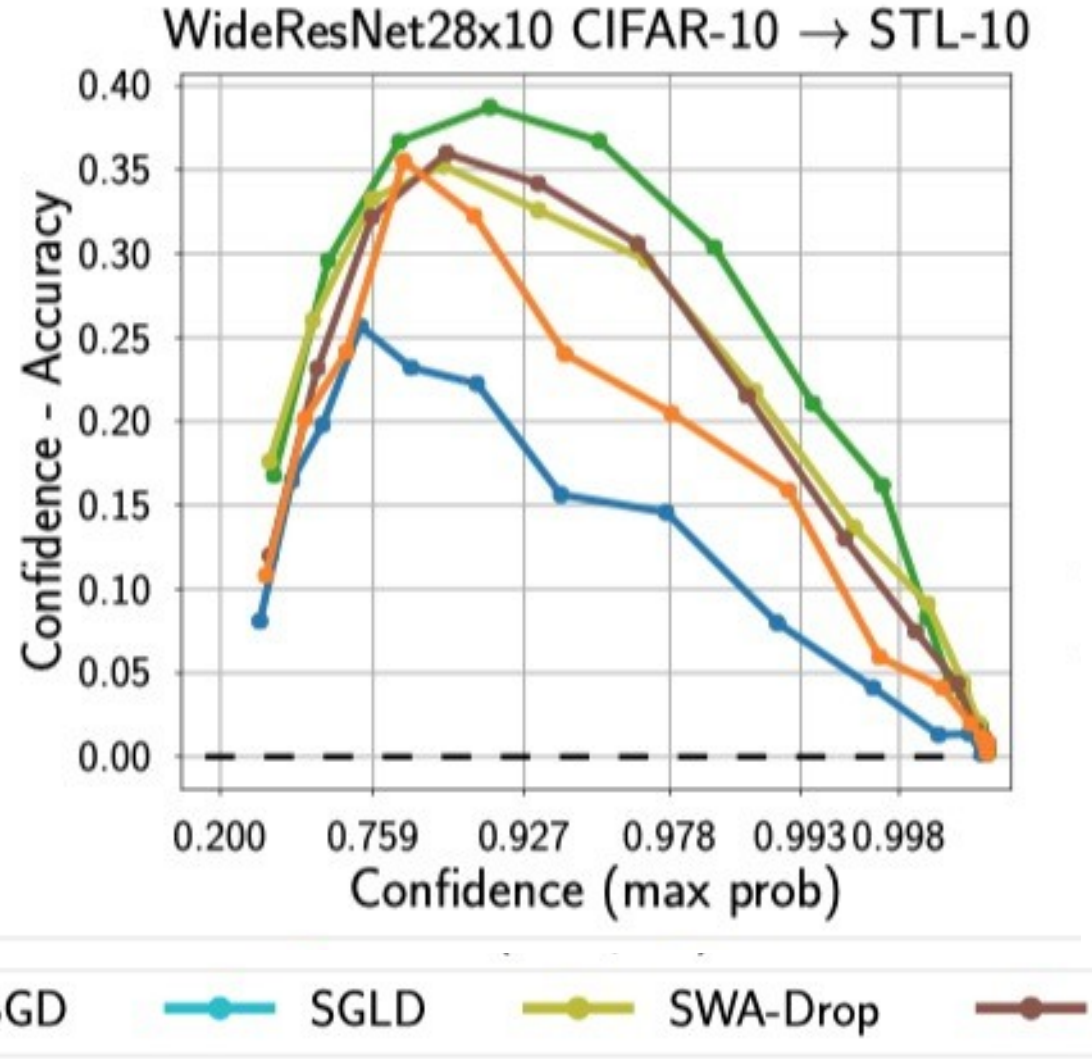
(a) True Labels (Err)



(b) True Labels (NLL)

Better confidence calibratic



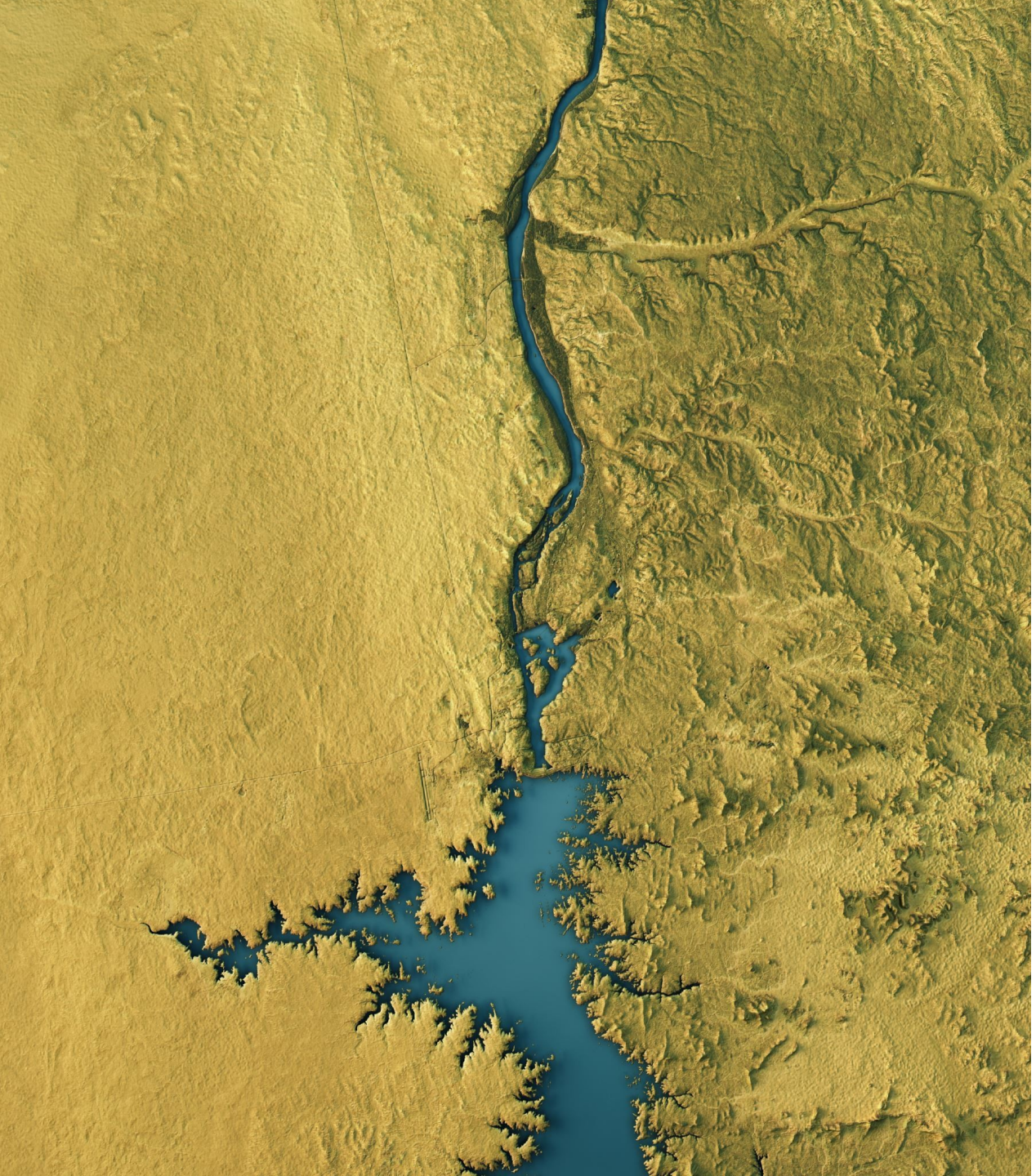


From SWAG paper.
This is a transfer learning setting where it does particularly well

I’ve also observed that Bayesian ensembling is quite effective for transfer learning in NLP

Drawbacks of Bayesian neural nets

- Computationally expensive (with varying levels of approximations)
- More assumptions – a prior, and a likelihood function for data
- Some things we do in neural nets seem hard to interpret in Bayesian way (e.g. data augmentation)



Loss landscapes and Bayesian posteriors

How loss connects to $p(\theta|D)$

For Bayesian learning

We want the posterior:

$$\log p(w|D) = \log p(D|w) + \log p(w) - \log Z(D)$$

Posterior | Likelihood function | Prior | Intractable normalizer

We can decompose in terms of the data points:

$$\log p(w|D) \propto \sum_i \log p(y_i|x_i, w) + \log p(w)$$

No 1/n here! If n is large, the effect of the prior is negligible

This is equivalent to our usual cross entropy loss (plus regularizer)

We can also say: $E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$ and we want to sample from $p(w|D) = e^{-E(w)}/Z$

Bayesian model aver

$$p(y|x,D) = \int p(y|x,w)p(w|D)dw$$

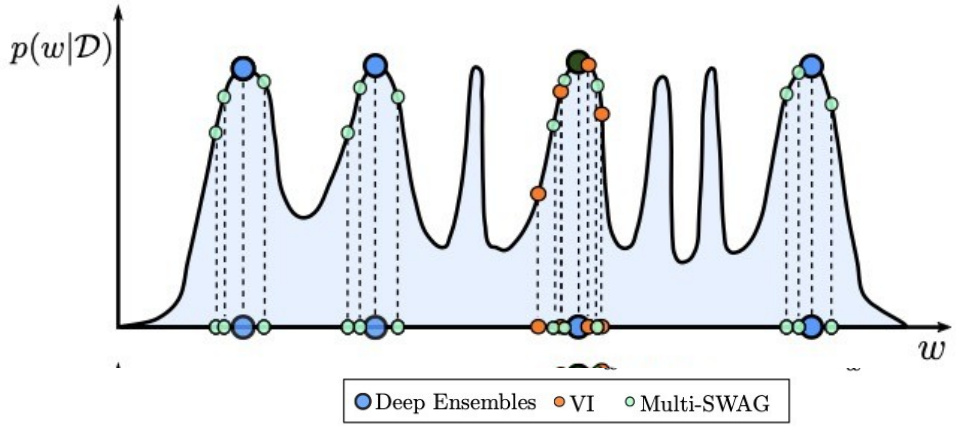


Figure 3. **Approximating the BMA.**

- Instead of doing the integral, we can approximate this integral by sampling w ~ p(w|D), then *average* the predictions, p(y|x,w).
- In other words, *ensemble* models, w, from the posterior

$$p(y|x,D) = E_{p(w|D)}[p(y|x,w)] \approx \frac{1}{n}\sum_i p(y|x,w_i)$$

$$\text{With } w_i \sim p(w|D)$$

- How do we sample from p(w|D) ???????

Bayesian Model Averaging

$$p(y|x,D) = \int p(y|x,\theta)p(w|D)d\theta$$
$$= E_{p(w|D)}[p(y|x,w)] \approx \frac{1}{n}\sum_i p(y|x,w_i)$$
$$\text{With } w_i \sim p(w|D)$$

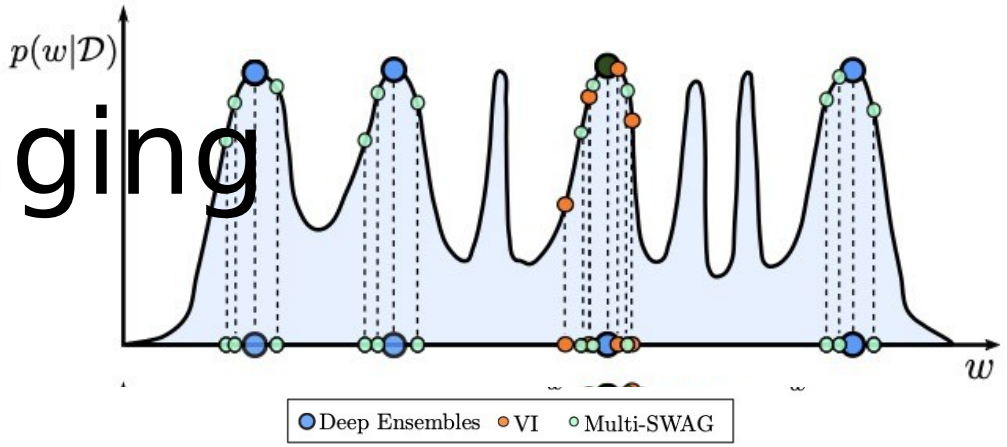


Figure 3. **Approximating the BMA.**

- Sample multiple solutions
  - MCMC dynamics
  - “Fast Ensembling” (approximate methods – will discuss)
- Learn a variational approximation of p(w|D)
  - E.g. Laplace approximation
- Do sampling / variational approximation in a lower-d subspace
- Kernel methods (I am not qualified… see Gordon tutorial)

Bayesian deep learning

Several things that could mean

And one or two that might actually be practical

I'm stealing several slides from

Roger Grosse's excellent course on "Neural Network Training Dynamics" (NNTD) https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2021/

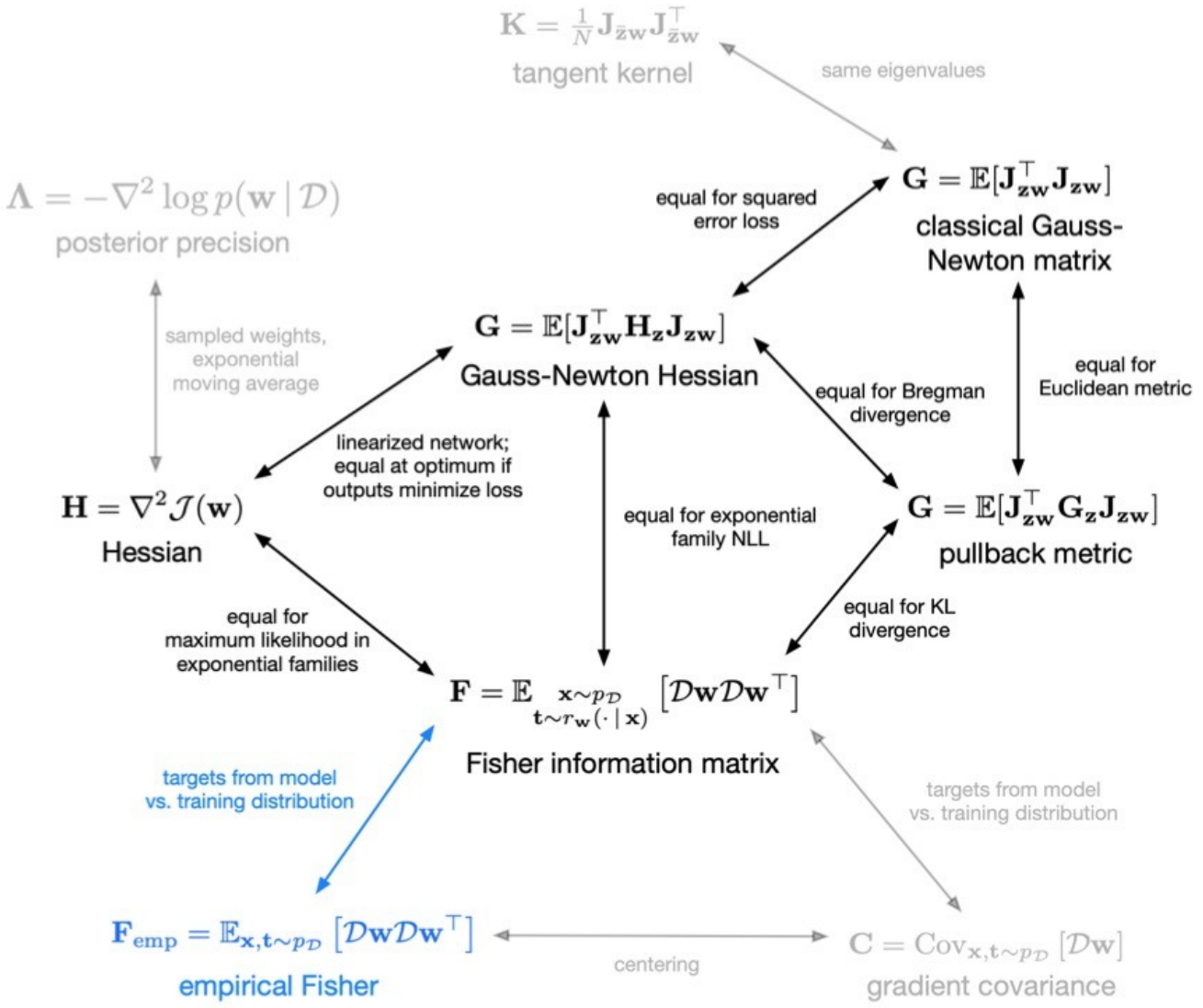


Figure 1: A summary of the relationships between the matrices used in this course. New items are in blue, and items yet to be covered are grayed out.

BNN Posterior Inference

- The cheapest and simplest way of training Bayesian neural nets is probably maximum a-posteriori (MAP) inference, which simply maximizes the posterior probability:

$$\begin{aligned}\mathbf{w}_{\text{MAP}} &= \arg\max_{\mathbf{w}} \log p(\mathbf{w} \mid \mathcal{D}) \\ &= \arg\max_{\mathbf{w}} \log p(\mathbf{w}, \mathcal{D}) \\ &= \arg\max_{\mathbf{w}} \overbrace{\sum_i \log p(\mathbf{t}^{(i)} \mid \mathbf{w}, \mathbf{x}^{(i)})}^{\text{likelihood}} + \overbrace{\log p(\mathbf{w})}^{\text{prior}}\end{aligned}$$

- With an i.i.d. Gaussian prior on the weights, the prior term is equivalent to ℓ_2 regularization (weight decay)

Bayesian DL could just mean: add a prior that functions as a regularizer while you optimize your network

We saw earlier that the MAP estimate can be bad – Bayesian model averaging

The Laplace approximation

- Optimize, then do a second order Taylor expansion around the optimum

$$\log p(w|D) \approx E(w^*) + (w - w^*)g(w^*) + (w - w^*)H(w^*)(w - w^*)$$

- To get uncertainty estimates, we can use the Laplace approximation to the posterior, which takes the second-order Taylor approximation to the log-likelihood around $\mathbf{w}_{\text{MAP}}$:

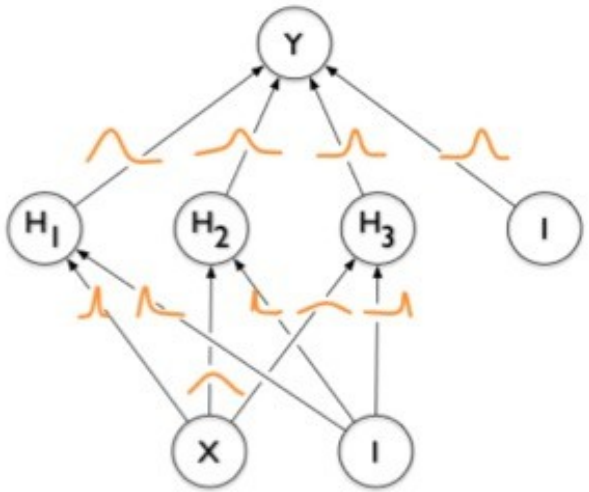
$$p(\mathbf{w} \mid \mathcal{D}) \approx \mathcal{N}(\mathbf{w}; \mathbf{w}_{\text{MAP}}, \bar{\mathbf{H}}^{-1})$$
$$\bar{\mathbf{H}} = N\mathbf{H} = -\nabla^2 \log p(\mathcal{D} \mid \mathbf{w})$$

- This is like the Hessian of the training cost, up to a factor of N

Variational BNNs

- The problem with the Laplace approximation is that uncertainty isn't considered until training is finished
- Variational Bayes is a more accurate posterior inference method which accounts for uncertainty during training
- We approximate a complicated posterior distribution with a simpler variational approximation. E.g., assume a Gaussian posterior with diagonal covariance (i.e. fully factorized Gaussian):

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$
$$= \prod_{j=1}^{D} \mathcal{N}(\theta_j; \mu_j, \sigma_j)$$



- This means each weight of the network has its own mean and variance.

— Blundell et al., Weight uncertainty for neural networks

Posterior Inference: Variational Bayes

- The marginal likelihood is the probability of the observed data (targets given inputs), with all possible weights marginalized out:

$$p(\mathcal{D}) = \int p(\mathbf{w})\, p(\mathcal{D} \mid \mathbf{w})\, \mathrm{d}\mathbf{w}$$
$$= \int p(\mathbf{w})\, p(\{t^{(i)}\} \mid \{\mathbf{x}^{(i)}\}, \mathbf{w})\, \mathrm{d}\mathbf{w}.$$

- Analogously to VAEs, we define a variational lower bound:

$$\log p(\mathcal{D}) \geq \mathcal{F}(q) = \mathbb{E}_{q(\mathbf{w})}[\log p(\mathcal{D} \mid \mathbf{w})] - \mathrm{D}_{\mathrm{KL}}(q(\mathbf{w}) \,\|\, p(\mathbf{w}))$$

- Unlike with VAEs, $p(\mathcal{D})$ is fixed, and we are *only* maximizing $\mathcal{F}(q)$ with respect to the variational posterior q (i.e. a mean and standard deviation for each weight).

Posterior Inference: Variational Bayes

- Likelihood term:

$$\mathbb{E}_{q(\mathbf{w})}[\log p(\mathcal{D} \mid \mathbf{w})] = \mathbb{E}_{q(\mathbf{w})}\left[\sum_{i=1}^{N} \log p(t^{(i)} \mid \mathbf{x}^{(i)}, \mathbf{w})\right]$$

  This is just the usual likelihood term (e.g. minus classification cross-entropy), except that $\mathbf{w}$ is sampled from q.

- KL term:

$$\mathrm{D}_{\mathrm{KL}}(q(\mathbf{w}) \,\|\, p(\mathbf{w}))$$

  This term encourages q to match the prior, i.e. each dimension to be close to $\mathcal{N}(0, \eta^{1/2})$.

- Without the KL term, the optimal q would be a point mass on $\mathbf{w}_{\mathrm{ML}}$, the maximum likelihood weights.
  - Hence, the KL term encourages q to be more spread out (i.e. more stochasticity in the weights).

Posterior Inference: Variational Bayes

- We can train a variational BNN using the same reparameterization trick as from VAEs.

$$\theta_j = \mu_j + \sigma_j \epsilon_j,$$

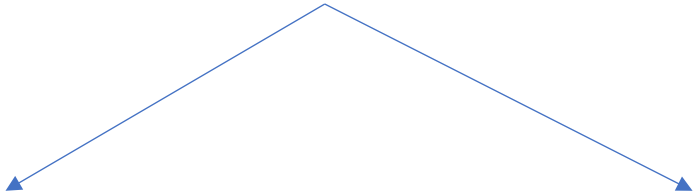
  where $\epsilon_j \sim \mathcal{N}(0, 1)$.
- Then the ϵ_j are sampled at the beginning, independent of the μ_j, σ_j, so we have a deterministic computation graph we can do backprop on.
- If all the σ_j are 0, then $\theta_j = \mu_j$, and this reduces to ordinary backprop for a deterministic neural net.
- Hence, variational inference injects stochasticity into the computations. This acts like a regularizer, just like with dropout.
  - The difference is that it's stochastic activations, rather than stochastic weights.
  - See Kingma et al., "Variational dropout and the local reparameterization trick", for the precise connections between variational BNNs and dropout.

Variational Bayes - takeaway

Clever approach that didn’t work well in practice

- Too noisy / hard to train
- Variational approximation used is not good enough
- Simpler approaches work better

One of these question marks is necessary



Practical(?) Bayesian(?) DL

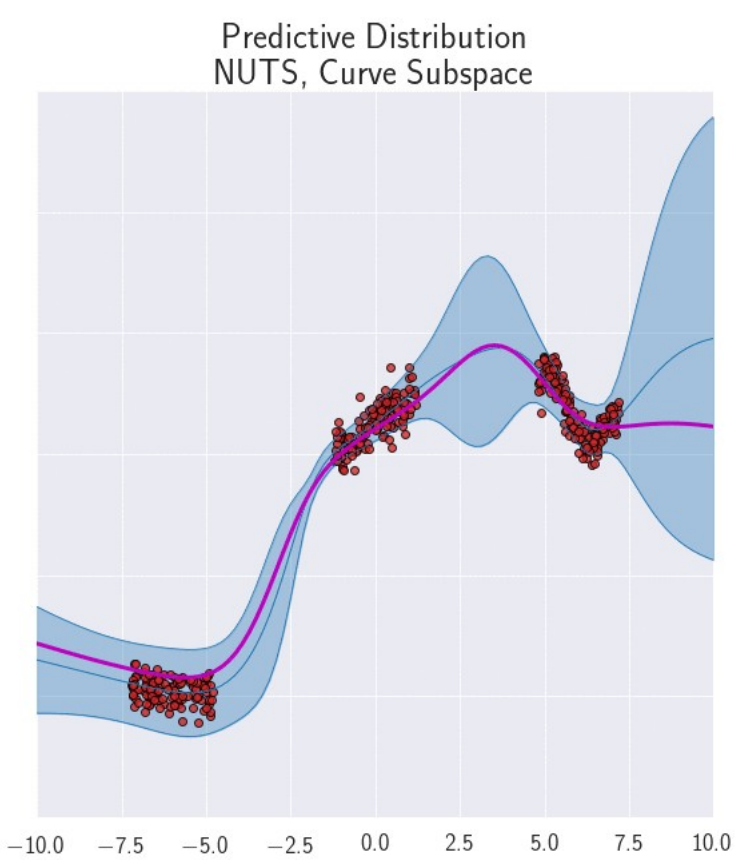
Sampling

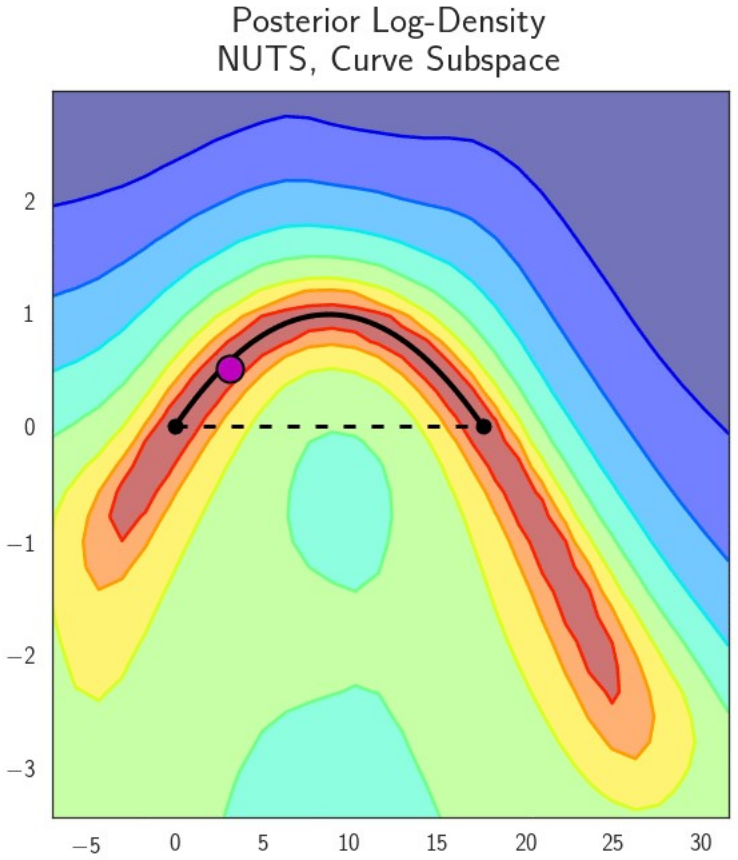
Fast Ensembling

Dropout

Noisy SGD

Sample in a low energy subspace?



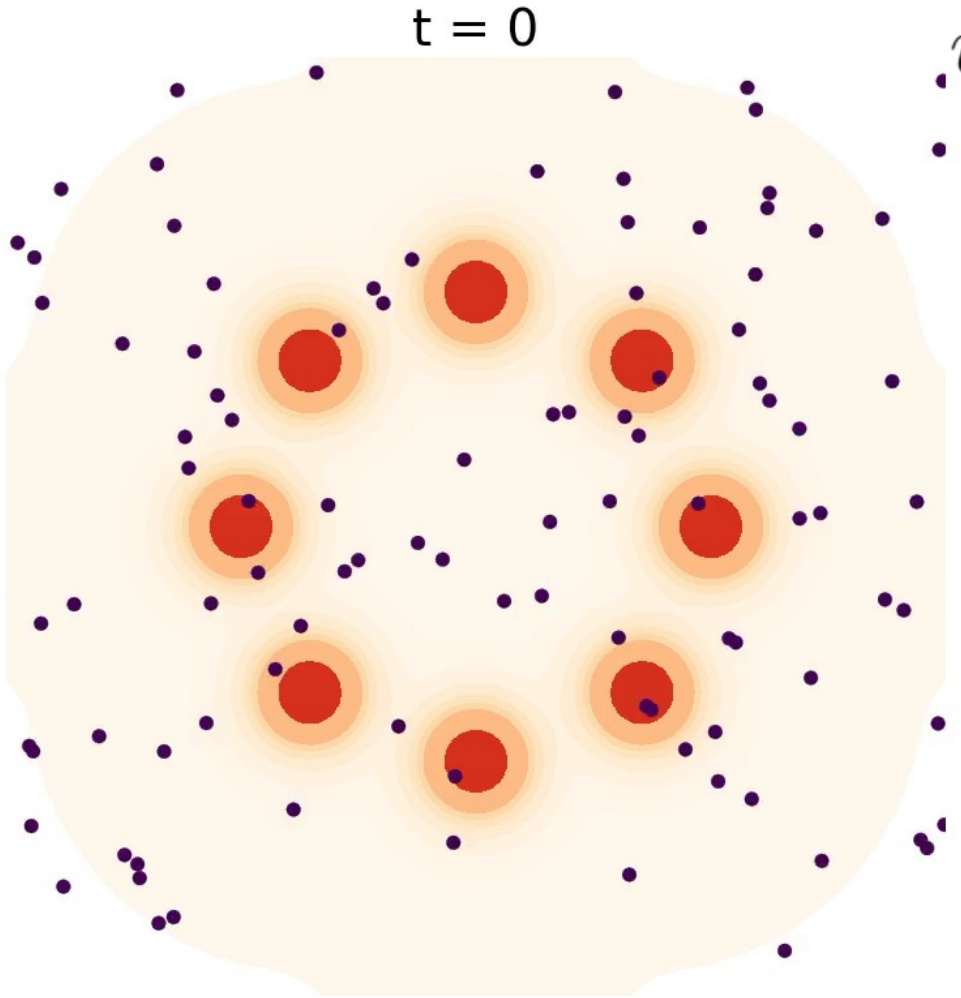


Sampling $p(w|D) = e^{-E(w)}/Z$

- With the energy defined:

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Langevin dynamics*



Weight space

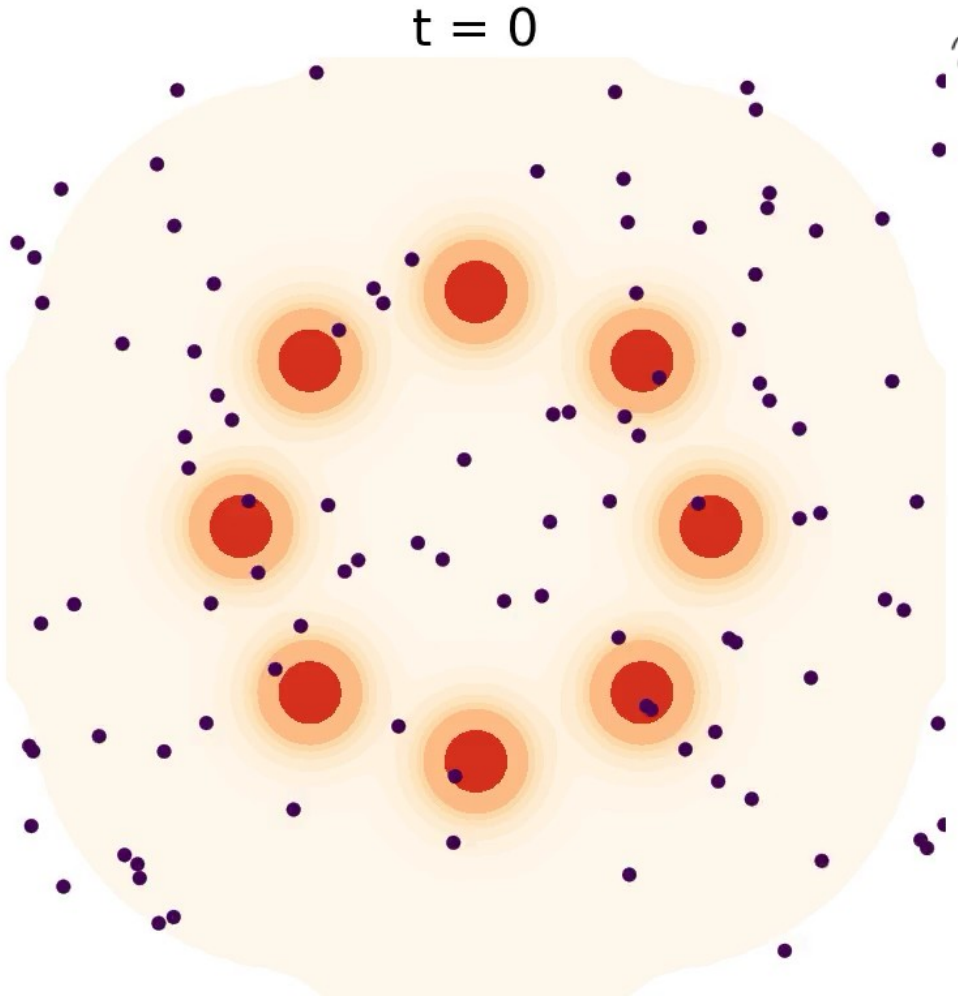
$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Gradient step toward low energy / high probability

Noise to explore $v \sim \mathcal{N}(0, \mathbb{I})$

**Technically, must add Metropolis-Hastings rejection step to make it a valid sampler, or take step size to zero, which no one does*

Problem Solved? Not so fast



Weight space

$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Require FULL BATCH backprop for each step!!!

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Sampling with mini-batches papers

- **Bayesian learning via stochastic gradient Langevin dynamics.** Welling, M., Teh, Y. ICML 2011
- Stochastic gradient Hamiltonian Monte Carlo. Chen, T., Fox, E., Guestin, C. ICML 2014
- Cyclical stochastic gradient MCMC for Bayesian deep learning. Zhang et. al, ICLR 2020

Welling & Teh, Stochastic Gradient Langevin Dynamics

$$w_{t+1} - w_t = -\eta\, g_{minibatch}$$

- From central limit theorem, we say that:

$$g_{minibatch} = \frac{1}{b} \sum_{j \in batch} g_j$$

$$g_{minibatch} \sim N(\bar{g}, \frac{\Sigma}{b})$$

$$w_{t+1} - w_t = -\eta\, g_{minibatch} = -\eta\, (\bar{g} + \frac{\Sigma^{\frac{1}{2}}}{\sqrt{b}}\, \epsilon)$$

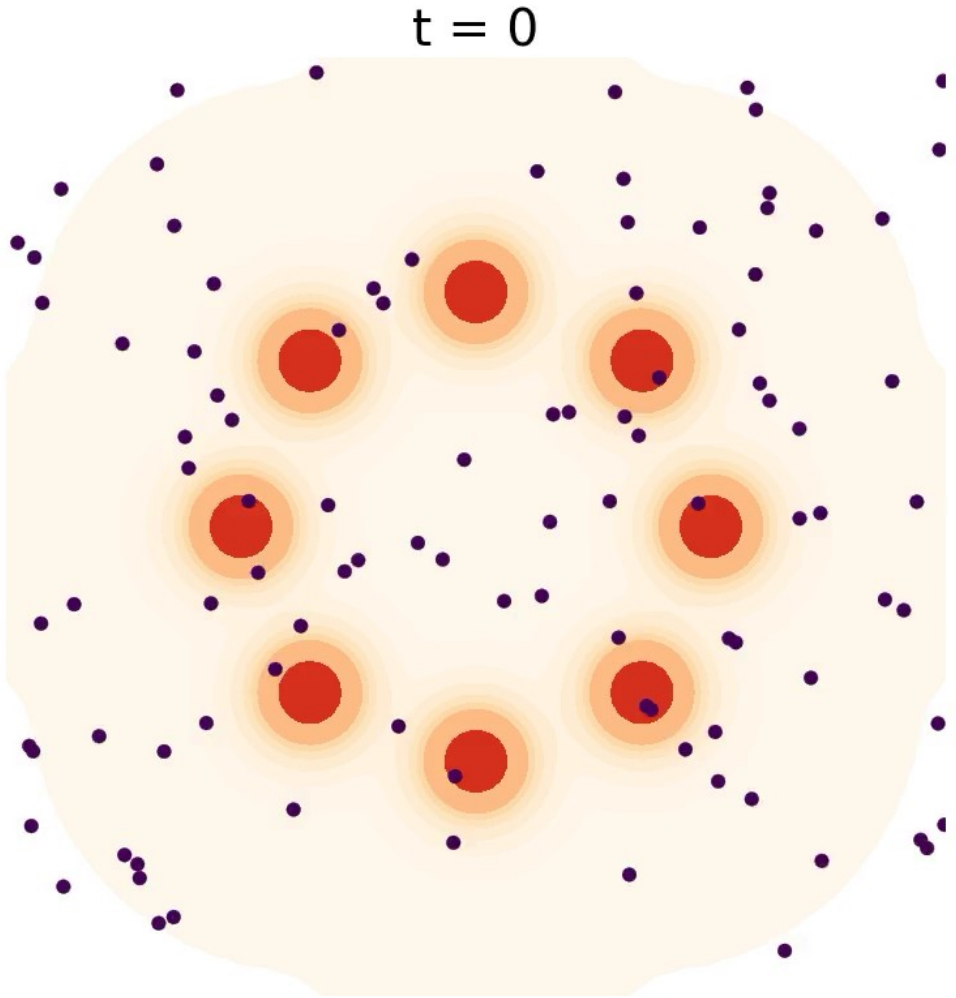
If we identify $\mathrm{g}(w) = \bar{g}, C = \sqrt{\frac{\eta}{b}\Sigma}$

$$w_{t+1} - w_t = -g(w_t)\eta + C(w_t)\, \sqrt{\eta}\, \epsilon$$

Problem

Let the step size, eta, go to zero, while adding a small amount of spherical Gaussian noise

Problem Solved?

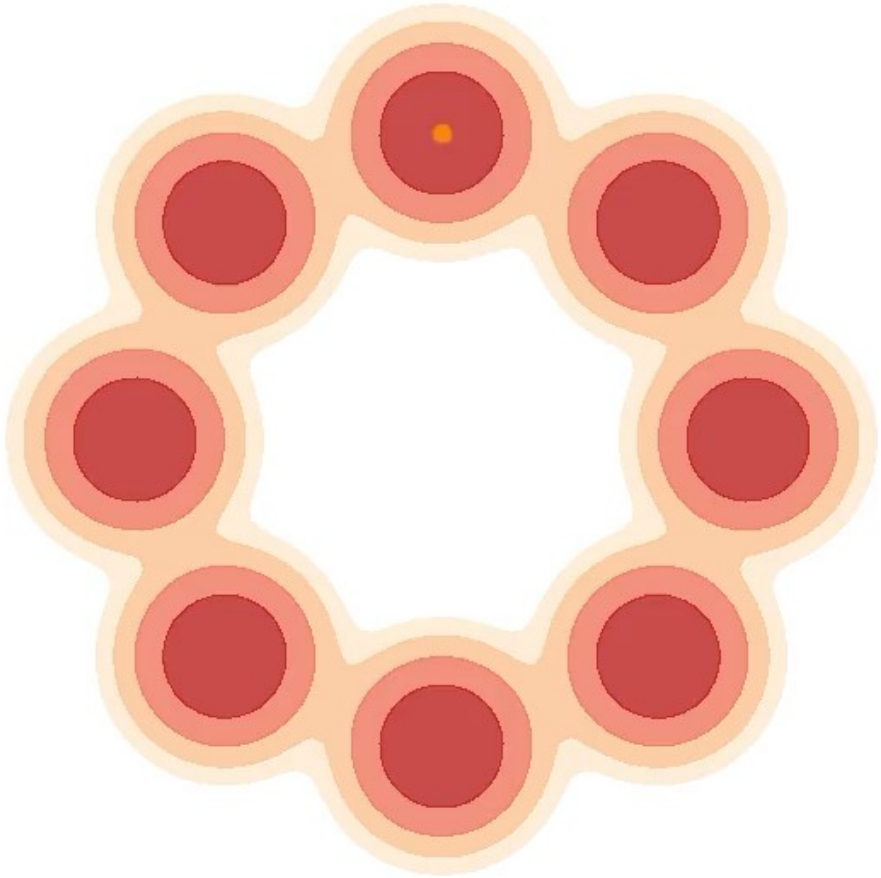


$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Use small learning rate, stochastic gradient, and add some gaussian noise

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Interesting observation... if the dynamics *kind of* sample weights over time... maybe instead of multiple samples we could look at one trajectory over time



Sampling one trajectory, over time

Langevin dynamics

ESH dynamics

(from my 2021 Neurips paper on “Non-Newtonian Momentum)

Width corresponds to time spent in each region (in unscaled time)

(New papers call this “microcanonical HMC”)

Not Bayesian – but averaging samples over time (Stochastic Weight Averaging) really seems to work!

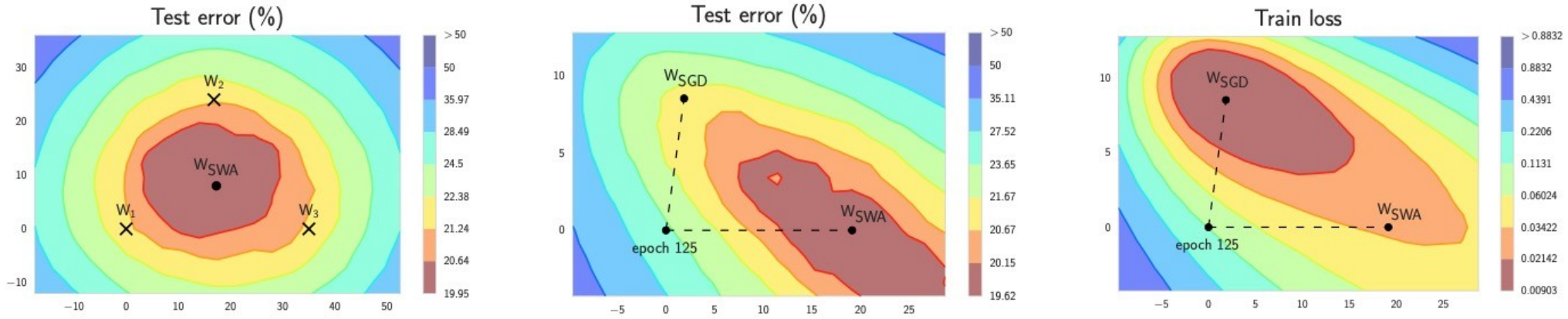


Figure 1: Illustrations of SWA and SGD with a Preactivation ResNet-164 on CIFAR-100[1]. **Left**: test error surface for three FGE samples and the corresponding SWA solution (averaging in weight space). **Middle** and **Right**: test error and train loss surfaces showing the weights proposed by SGD (at convergence) and SWA, starting from the same initialization of SGD after 125 training epochs.

Lots of weight averaging approaches
Federated learning, Polyak averaging

Fast ensembling (Stochastic Weight Averaging)

Algorithm 1 Stochastic Weight Averaging

Require:
weights $\hat{w}$, LR bounds α_1, α_2,
cycle length c (for constant learning rate $c = 1$), number of iterations n

Ensure: w_{SWA}

$w \leftarrow \hat{w}$ {Initialize weights with $\hat{w}$}
$w_{\text{SWA}} \leftarrow w$
for $i \leftarrow 1, 2, \ldots, n$ **do**
  $\alpha \leftarrow \alpha(i)$ {Calculate LR for the iteration}
  $w \leftarrow w - \alpha \nabla \mathcal{L}_i(w)$ {Stochastic gradient update}
  if $\mathrm{mod}(i, c) = 0$ **then**
    $n_{\text{models}} \leftarrow i/c$ {Number of models}
    $w_{\text{SWA}} \leftarrow \frac{w_{\text{SWA}} \cdot n_{\text{models}} + w}{n_{\text{models}} + 1}$ {Update average}
  end if
end for
{Compute BatchNorm statistics for w_{SWA} weights}

SWAG

A Simple Baseline for Bayesian Uncertainty in Deep Learning

Algorithm 1 Bayesian Model Averaging with SWAG

θ_0: pretrained weights; η: learning rate; T: number of steps; c: moment update frequency; K: maximum number of columns in deviation matrix; S: number of samples in Bayesian model averaging

Train SWAG

$\overline{\theta} \leftarrow \theta_0, \;\; \overline{\theta^2} \leftarrow \theta_0^2$ {Initialize moments}

for $i \leftarrow 1, 2, ..., T$ **do**

$\theta_i \leftarrow \theta_{i-1} - \eta \nabla_\theta \mathcal{L}(\theta_{i-1})$ {Perform SGD update}

if $\texttt{MOD}(i, c) = 0$ **then**

$n \leftarrow i/c$ {Number of models}

$\overline{\theta} \leftarrow \frac{n\overline{\theta} + \theta_i}{n+1}, \;\; \overline{\theta^2} \leftarrow \frac{n\overline{\theta^2} + \theta_i^2}{n+1}$ {Moments}

if $\texttt{NUM\_COLS}(\widehat{D}) = K$ **then**

$\texttt{REMOVE\_COL}(\widehat{D}[:, 1])$

$\texttt{APPEND\_COL}(\widehat{D}, \theta_i - \overline{\theta})$ {Store deviation}

return $\theta_{\text{SWA}} = \overline{\theta}, \;\; \Sigma_{\text{diag}} = \overline{\theta^2} - \overline{\theta}^2, \;\; \widehat{D}$

Test Bayesian Model Averaging

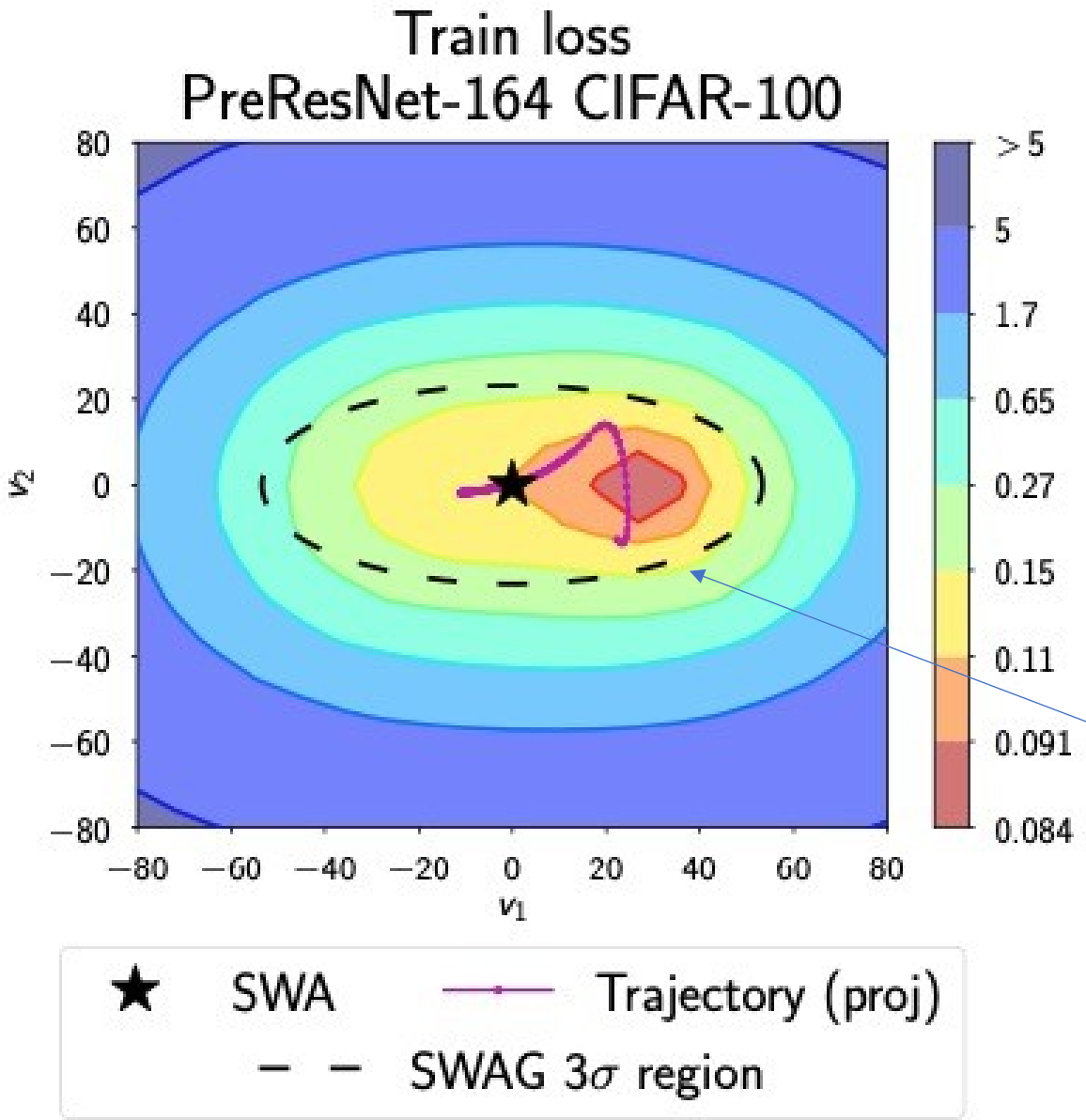
for $i \leftarrow 1, 2, ..., S$ **do**

Draw $\widetilde{\theta}_i \sim \mathcal{N}\left(\theta_{\text{SWA}}, \frac{1}{2}\Sigma_{\text{diag}} + \frac{\widehat{D}\widehat{D}^\top}{2(K-1)}\right)$ (1)

Update batch norm statistics with new sample.

$p(y^*|\text{Data}) \mathrel{+}= \frac{1}{S} p(y^*|\widetilde{\theta}_i)$

return $p(y^*|\text{Data})$



Sample solutions from this learned approximate Gaussian, for Bayesian model averaging